4 Conhecendo o R

AUTOR
Artur Damião

AFILIAÇÃO
Programa de Pós-Graduação em Sociologia (USP)

DATA DE PUBLICAÇÃO
2 de fevereiro de 2026

4.1 Informações gerais

- **Data/horário:** 02 a 06 de fevereiro/2026;
  - Todos os dias da semana: aula ao vivo, on-line, das 18h00 às 20h00.
  - Todo estudo adional é encorajado.

4.2 As vantagens de analisar dados usando linguagens de programação

- O código é uma linguagem, então podemos documentar os passos da nossa análise. Isso é, escrever, linha à linha, os passos do processo de análise.
- O código é texto, então podemos copiar e colar, facilitando enormemente a transposição do código para novos casos.
- As linguagens de programação para ciência de dados mais utilizadas são de código aberto, em especial R, com dominância em certos nichos acadêmicos, e Pyhton, com popularidade ascentende e mais utilizado no universo corporativo.
  - Fora do Brasil, é mais comum empresas de ponta utilizarem o `R` ao invés de Python.
- Alternativas proprietárias (fechadas, controladas por empresas que cobram por licenças ou assinatura) incluem softwares como Stata, SAS, SPSS, entre outras. Ou seja, mais **custo** para o negócio.

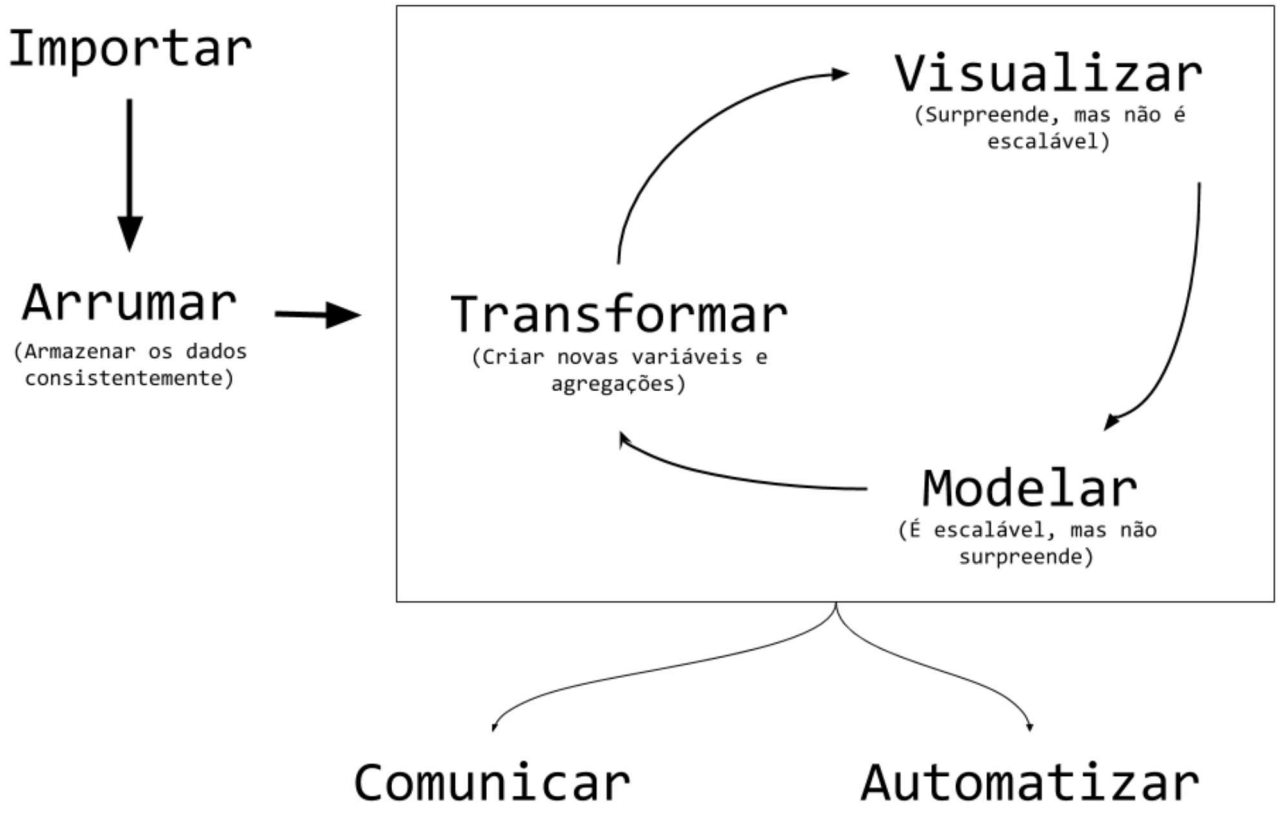
4.3 O que significa ser código aberto e software livre?

1. *Free Software* como em “liberdade” não como em “grátis”. Na prática temos acesso gratuito (na maioria dos casos). Os usuários podem modificar e aperfeiçoar o código-fonte.
2. As pessoas podem acessar as ferramentas, tendo as condições mínimas em termos de *hardware* e conhecimento, independente de poderem pagar uma licensa de uso.

3. Estudantes podem usar software semelhante ao de profissionais.
4. Você pode corrigir problemas e aprimorar a linguagem e eventualmente contribuir e modificar o projeto principal.
5. Você pode desenvolver suas próprias ferramentas.
6. Possibilita a existência de uma comunidade ativa.

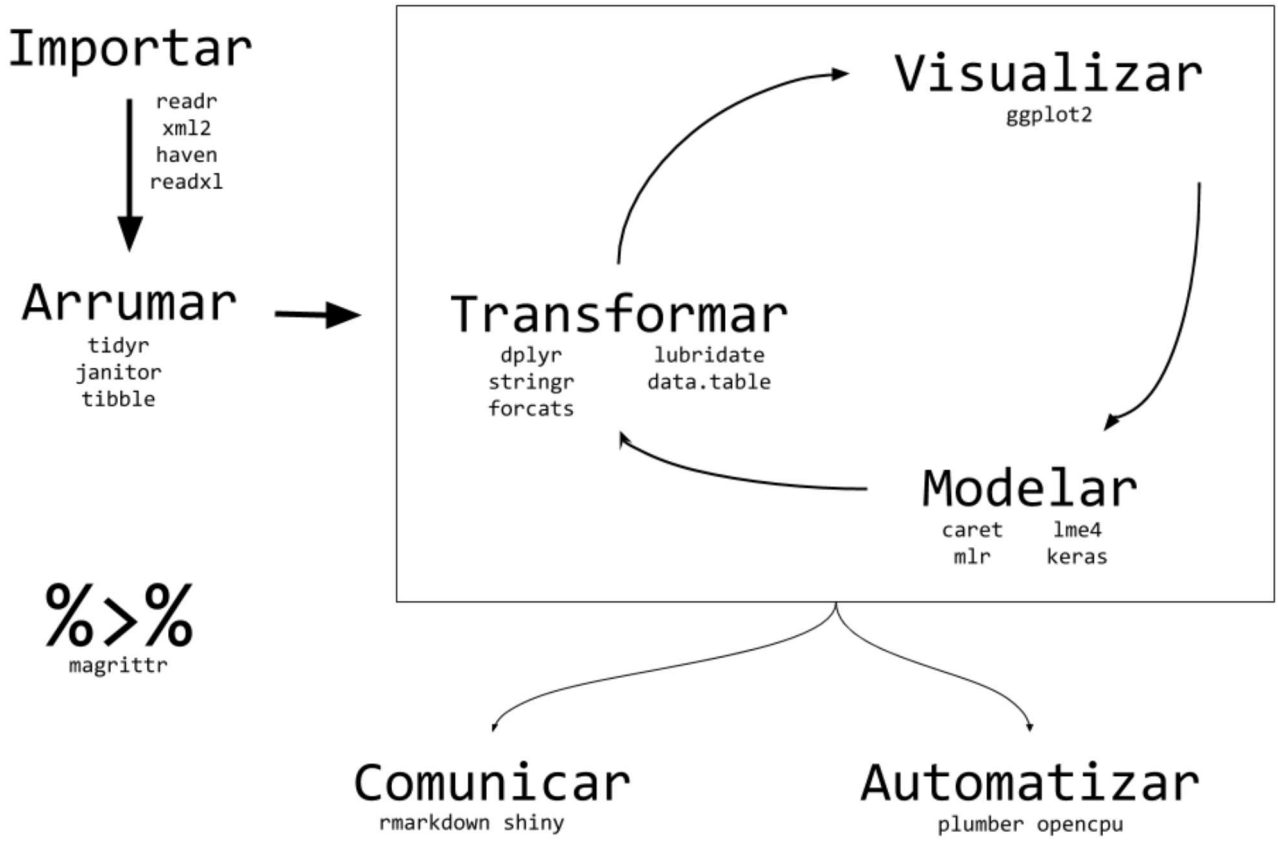
4.4 Por quê o R?

4.4.1 O ciclo da ciência de dados



O ciclo da Ciência de Dados

4.4.2 O ciclo da ciência de dados com o R



O ciclo da Ciência de Dados no R

4.5 Por quê usar o R?

- É uma linguagem de programação que possui muitas ferramentas para análise de dados
- É *código aberto* (open source)
- Possui uma comunidade ativa de pessoas desenvolvedoras
- É flexível, permite desenvolver funções e pacotes para facilitar o trabalho
- Está disponível, gratuitamente, em diferentes plataformas: Windows, Linux e Mac
- Mantido pela R Development Core Team

5 Reprodutibilidade

5.1 Um exemplo prático e cotidiano

- Vocês já tentaram repetir uma análise feita meses atrás e não lembraram como chegaram no resultado?
- Já recebeu uma planilha e não entendeu como a pessoa chegou naqueles resultados, já que faltam passos claros de como executar a análise?
- Você já ficou “irritado” de ter que fazer toda uma análise, que parece MUITO com alguma outra que você já fez mas que só mudou a variável de interesse?

- Isso conecta com a experiência comum de quem lida com dados;
- Reprodutibilidade é como pensar no seu eu do futuro e na sua equipe: todo mundo deve ser capaz de apertar *Run* e obter exatamente o mesmo resultado.

5.2 Por que isso é importante?

- **Validação do conhecimento**: Permite confirmar resultados e aumentar a credibilidade científica;
- **Transparência e confiança**: Exige documentação clara de métodos, dados e códigos;
- **Acúmulo de conhecimento**: Garante que novas pesquisas possam se apoiar em resultados anteriores;
- **Correção de erros e vieses**: Facilita a detecção de falhas metodológicas ou fraudes;
- **Formação de analistas e pesquisadores**: Reproduzir estudos é parte essencial do aprendizado científico;

5.3 Ok e qual a relação com o curso de `R`?

- Queremos mostrar que aprender `R` não é só "programar", mas se alinhar com boas práticas;
- Usar `R` significa estar preparado para dialogar com transparência em todos os ambientes;
- Vamos aprender a preparar scripts que visam documentar **todo o processo**.

5.4 Por que a reprodutibilidade é fundamental?

5.4.1 Em resumo

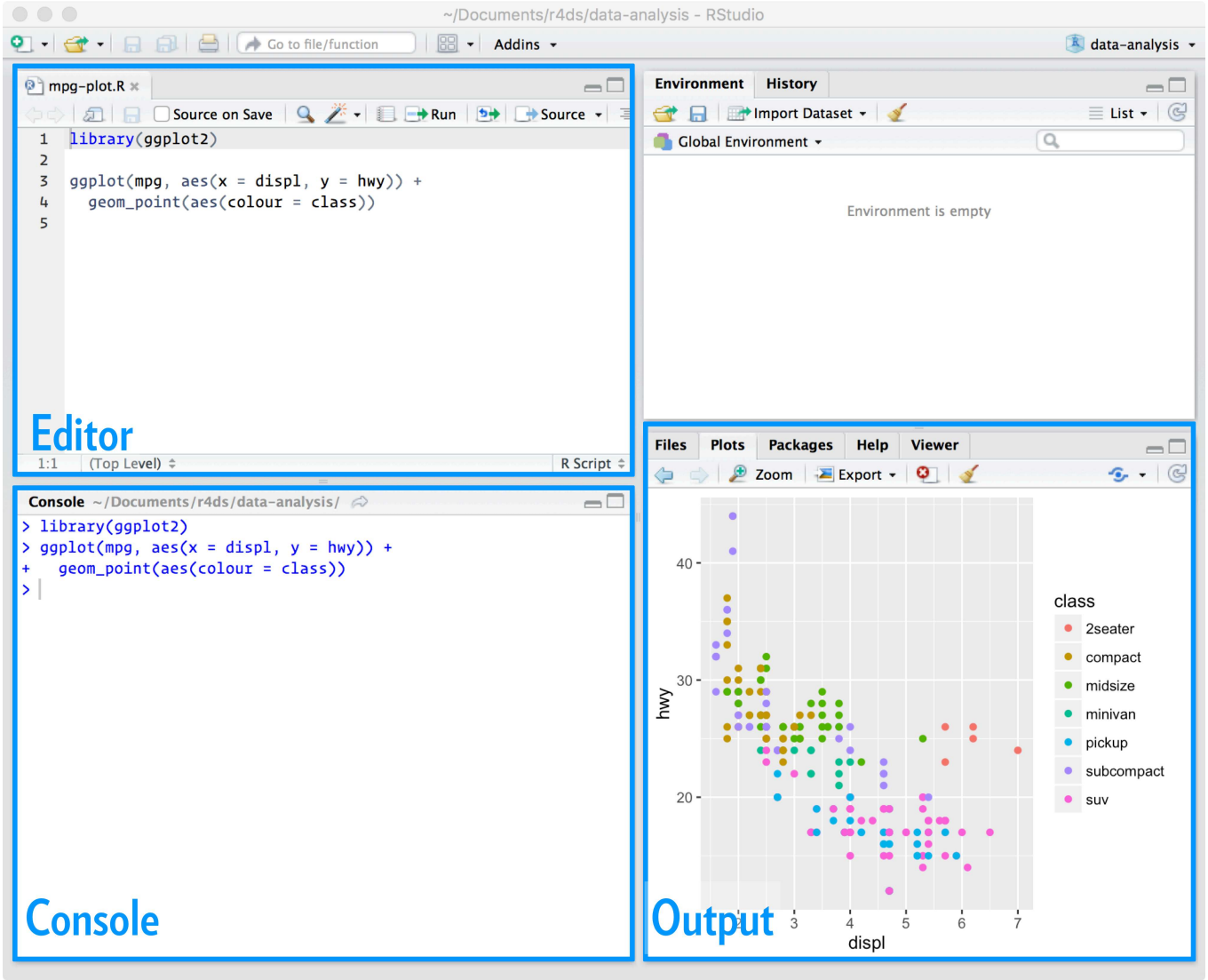
> A reprodutibilidade é um **pilar ético e epistemológico da ciência**, pois assegura que o conhecimento seja verificável e coletivo, não apenas dependente da autoridade de um pesquisador.

5.5 Plano das aulas

- Temas:
  - Aula 01: Apresentação e motivação do curso
    - Configuração e Ambientação dentro do RStudio
    - Primeiros passos e conceitos fundamentais
  - Aula 02:Manipulação de dados, parte 1
  - Aula 03:Manipulação de dados, parte 2
  - Aula 04: Visualização com `{ggplot2}`
    - Relatórios dinâmicos com `Quarto`
  - Aula 05: Mapas e variáveis demográficas

6 Configuração e ambientação do RStudio

6.1 Conhecendo o Rstudio



O Rstudio

- O RStudio apresenta 04 quadrantes:
  - Editor/Script: é onde passaremos a maior parte do tempo.
  - Console: é onde os códigos são "enviados" e onde obtemos as respostas. Este é o `R`!
  - Os outros painéis são auxilares:
    - Environment: painel com os `objetos` criados na sessão;
    - History: painel com o histórico de comandos executados;
    - Files: é como se fosse a Área de Trabalho do seu computador. Mostra os arquivos;
    - Plots: autoexplicativo. É onde as imagens geradas aparecem;
    - Packages: janela para encontrar a documentação dos pacotes e funções;
    - Viewr: Onde relatórios e dashboards aparecem.

6.2 Atenção!

> **Aviso**
>
> O `RStudio` é um pai coruja 🦉. Isso significa que ele vai querer te lembrar de **TODOS** os comandos que você executou anteriormente. Isso é salvo em dois arquivos ocultos: `.RData` (guarda os objetos de uma sessão) e `.Rhistory` (abriga todos os comandos executados).

Qual o problema disso, então?

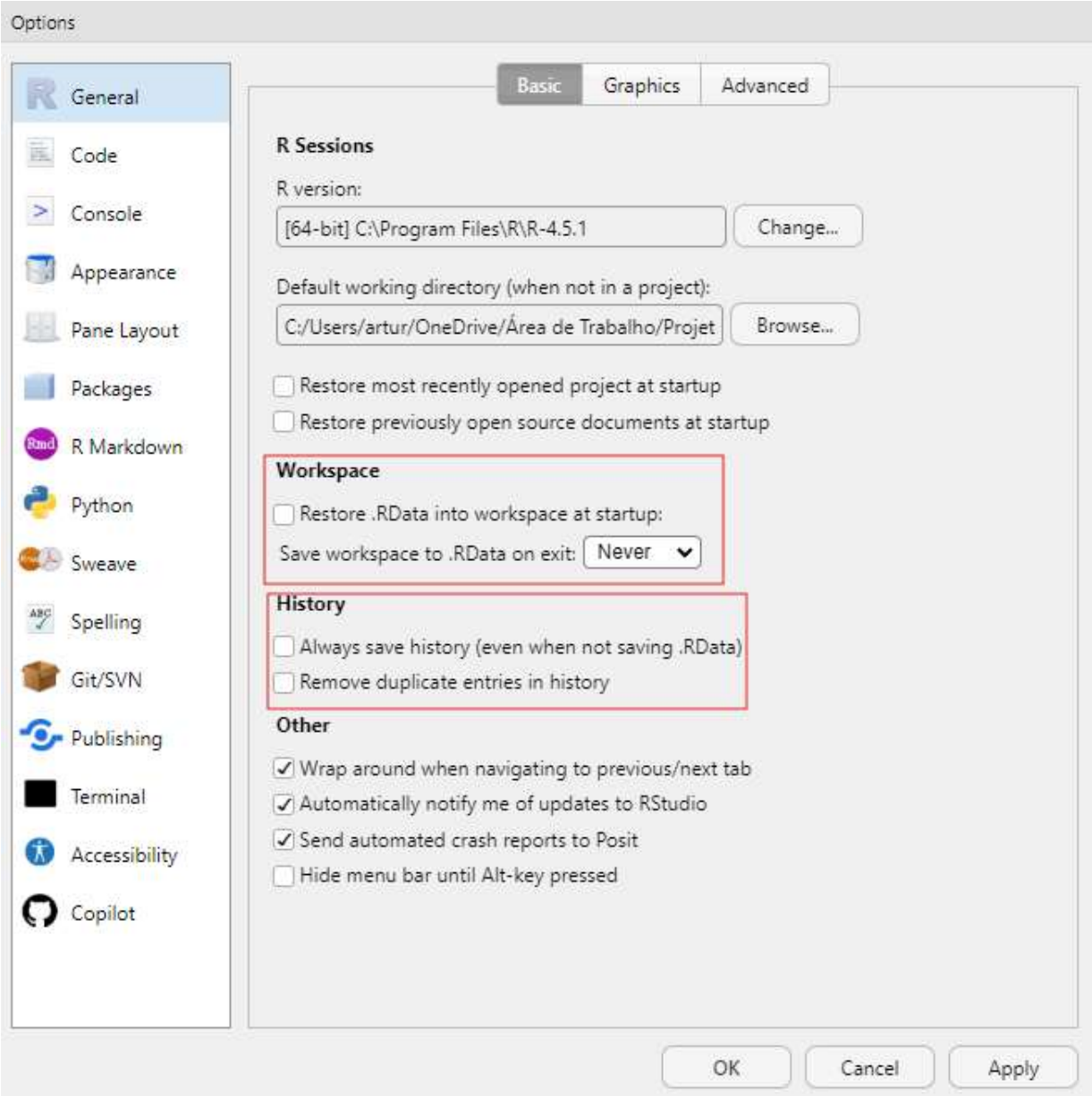
1. Te acostuma mal
2. Faz com que você conte com sua sorte! ☘️
3. Dificulta a **reprodutibilidade**

i. Afinal, você não precisa se preocupar em fazer um código legível, porque sempre contará com a facilidade de ter os objetos e comandos salvos.

4. O uso do `.RData` pode fazer com que seu ambiente de trabalho fique muito pesado!

i. Isso porque ele vai salvar todas as suas bases de dados, independente do tamanho.
ii. Logo, você terá muito mais lentidão ao iniciar o `RStudio`, dependendo das configurações de seu computador.

6.3 Como desativar?[1]



:

- Vá em `Tools -> Global options... -> General`
- Pronto! Agora você vai garantir um ambiente de trabalho eficiente e reprodutível.

> Se acostumar com sessões efêmeras não é uma tarefa fácil e um desconforto inicial é esperado. Pode ser que você ache entediante executar o mesmo código toda vez que abrir seu RStudio, mas é importante ter em mente que esse programa só está lá e pode ser executado inúmeras vezes porque o R não estará mais fazendo o **seu** trabalho. *Zen do R*, capítulo 2

[1] Atenção: se você nunca usar, nem vai sentir falta!

7 Direto para o `R`!

7.1 O Básico do `R`

Vamos para o quadrante do `Editor/Script` no RStudio. Se somarmos, por exemplo, `1 + 1`, veremos que o `R` nos retorna `[1] 2`.

```
1 + 1
```

```
[1] 2
```

```
# Se você usar uma "#", o código fica comentado. Ou seja, é só um texto!

# 1 + 1
```

- Se colocarmos uma `#` antes do código, nós estamos comentando ele.
  - Assim, o código não é executado.
  - Comentar o código é bom para documentar a análise!
    - Afinal, a tendência é nós esquecermos as coisas. Portanto, uma boa prática é deixar o registro do que fizemos!
    - PS: os seus colegas podem se beneficiar disso também. 😉

7.2 `R` como calculadora

Todas as operações possíveis!

```
# adição
1 + 1
```

```
[1] 2
```

```
# subtração
4 - 2
```

```
[1] 2
```

```
# multiplicação
2 * 3
```

```
[1] 6
```

```
# divisão
5 / 3
```

```
[1] 1.666667
```

```
# potência
4 ^ 2
```

```
[1] 16
```

```
# resto da divisão de 10 por 3
10 %% 3
```

```
[1] 1
```

```
# parte inteira da divisão de 10 por 3
10 %/% 3
```

```
[1] 3
```

Assim como na matemática, a divisão e a multiplicação são calculadas antes da adição e da subtração! De qualquer forma, usar os parênteses nunca é demais.

```
50 + 1 * 8
```

```
[1] 58
```

É o mesmo que:

```
50 + (1 * 8)
```

```
[1] 58
```

E diferente de :

```
(50 + 1) * 8
```

```
[1] 408
```

> **Dica**
>
> No `Script`, ao final da linha de código, você pode executar `CTRL + ENTER`. Este é o atalho para **rodar, avaliar** ou **executar** o seu código! Você vera a resposta diretamente no **Console**.

7.3 Objetos, vetores e funções

7.3.1 Objetos

Grosso modo, objetos são nomes que guardam informações. Tabelas, números, funções etc. Para criar um objeto, basta usar os operadores `<-` ou `=`.

```
x <- 1 # Leia-se: atribuo o valor 1 ao objeto x
y = 2  # Leia-se: atribuo o valor 2 ao objeto y

X <- 2
```

> **Dica**
>
> Na comunidade `R`, a convenção é usarmos o sinal `<-` ao invés do sinal de `=`. Na maioria dos casos, se você usar `<-` ou `=`, vai obter o mesmo resultado. Entretanto, o sinal de igualdade é reservado para `funções` dentro do `R`. O atalho para o sinal de atribuição `<-` é `ALT -`.

7.3.2 O `R` é sensível 😭 😱

7.3.2.1 O que isso quer dizer?

O `R` diferencia letras maiúsculas e minúsculas. Tecnicamente, dizemos que o `R` é *case sensitive*.

```
x
```

```
[1] 1
```

```
# é diferente de

X
```

```
[1] 2
```

```
meu_objeto <- c(1:10)

Meu_objeto <- c(11:20)
```

```
meu_objeto
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
Meu_objeto
```

```
 [1] 11 12 13 14 15 16 17 18 19 20
```

7.4 Boas práticas de “nomenclatura”

1. Evitar o uso de MAIÚSCULAS
2. Evitar o uso de nomes iguais a funções
   i. `mean()`, `sum()`, `sd()`
3. Evitar acentos ou caracteres especiais
   i. Evitar o uso de Ç

```
ação <- 1
```

4. Usar nomes já existentes no seu ambiente de trabalho, porque o `R` irá sobrescrever
5. Recomendamos:

```
bancodedados <- 1   # Ruim
banco_de_dados <- 1 # Bom
```

```
i. Ou seja, substituimos os ` ` por `_`
```

7.5 Vetores

Os objetos no `R` podem comportar muitas informações de uma só vez. A mais frequente que nós utilizamos são os **vetores**. Eles são um conjunto de informações atribuídas em uma única linha. Os vetores podem ser:

1. Numéricos:

```
vetor_1 <- 1:10 # Vetor que vai de 1 até 10
vetor_2 <- c(0.1, -5.0, 3.13, 100.1)
```

2. Textuais (conhecidos como `character`):

```
vetor_4 <- "Olá Mundo"
vetor_5 <- c("Conjunto", "de", "texto", "em", "um", "vetor")
```

É possível verificar a classe dos vetores:

```
class(vetor_1) # integer (numero inteiro)
```

```
[1] "integer"
```

```
class(vetor_2) # numeric (apresenta número decimal)
```

```
[1] "numeric"
```

```
class(vetor_5) # character (texto)
```

```
[1] "character"
```

7.6 Pedindo ajuda para o `R`

No slide anterior, vimos uma coisa estranha que ainda não apresentamos. O que é o `c()` ?

```
vetor_5 <- c("Conjunto", "de", "texto", "em", "um", "vetor")
```

Para pedir ajuda ao `R`, basta ir no `Console` e digitar:

```
?c
```

```
?mean
```

7.7 Manipulando objetos

```
# Criar o objeto
meu_objeto <- c(0:50, 60, 70, 80, 90)
meu_objeto
```

```
 [1]  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
[26] 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
[51] 50 60 70 80 90
```

```
# "Acessar" o valor de determinado elemento do vetor
meu_objeto[1] # acessa o 1o elemento do vetor 'meu_objeto'
```

```
[1] 0
```

```
meu_objeto[55] # acessa 55o elemento do vetor 'meu_objeto'
```

```
[1] 90
```

```
# Por meio dessa selecao de valores, podemos fazer operacoes:
meu_objeto[1] + meu_objeto[10] + meu_objeto[55]
```

```
[1] 99
```

```
meu_objeto[1] + meu_objeto[10] * meu_objeto[55]
```

```
[1] 810
```

```
meu_objeto[1] - meu_objeto[10]
```

```
[1] -9
```

```
meu_objeto[10] ^ meu_objeto[3]
```

```
[1] 81
```

7.7.1 Funções

```
# Calcular algumas estatisticas descritivas:
mean(meu_objeto) # media
```

```
[1] 28.63636
```

```
max(meu_objeto)  # maximo
```

```
[1] 90
```

```
min(meu_objeto)  # minimo
```

```
[1] 0
```

```
# Guardar essas informacoes em objetos
media  <- mean(meu_objeto)
maximo <- max(meu_objeto)
minimo <- min(meu_objeto)

# Unir em um vetor
## c significa concatenar
descritivas <- c(media, maximo, minimo)
descritivas
```

```
[1] 28.63636 90.00000  0.00000
```

7.8 Operadores lógicos

Operadores lógicos no `R`

Operador	Operação
`!`	Negação (não)
`&`	E (avalia elemento a elemento)
`|`	Ou (avalia elemento a elemento)
`<`	Menor
`<=`	Menor ou igual
`>`	Maior
`>=`	Maior ou igual
`!=`	Diferente
`&&`	E (avalia apenas o 1º elemento)

Operador	Operação
`||`	Ou (avalia apenas o 1º elemento)

8 Diretório de trabalhos e Projetos

- Alguns conceitos são importantes para se trabalhar no `R`. Um deles é a ideia de diretórios de trabalhos (*working directory*) e projetos.
    - Isso porque quando queremos importar algum arquivo em nosso computador (para fazer uma análise, por exemplo), precisamos dizer exatamente onde este arquivo se encontra.
- Para isso, usamos o conceito de `caminhos`.

8.0.1 Caminhos absolutos e relativos

`Caminhos` podem ser:

- Absolutos:

```
dados <- readxl::read_xlsx("C:/Users/artur/OneDrive/Área de Trabalho/Projetos/cursos-oferecido
```

Ou seja, o caminho completo desde a raiz do seu computador até o arquivo desejado.

- Relativos:

```
dados <- readxl::read_xlsx("dados/dados_brutos.xlsx")
```

Ou um caminho **relativo** à uma pasta de referência.

> **Boas práticas**
>
> Usar caminhos relativos, ao invés de caminhos absolutos, garante que o código funcione em **qualquer** computador!

8.1 Diretórios de trabalho

Um diretório de trabalho no `R` é o diretório (ou pasta) para onde o R se baseia quando precisamos interagir com os nossos arquivos.

- Para sabermos qual nosso diretóio atual, usa-se a função `getwd()`

```
getwd()
```

```
[1] "D:/OneDrive/Área de Trabalho/Projetos/cursos-oferecidos/2026-02-summer-intro-crime-
analysis-R/chapters"
```

8.2 Projetos no RStudio: o `.RProj`

A melhor forma de organizar os seus arquivos dentro do `R` é usando Projetos.

- Quando criamos um `projeto` no RStudio, ele cria uma pasta para nós e define essa pasta como diretório de trabalho (lembra do caminho relativo?).
  - Além disso, o RStudio também cria um arquivo chamado `.RProj` dentro da pasta.
  - Trabalhar com `projetos` facilita a organização de arquivos e a reprodução do código.
- Canto superior esquerdo, clicar em `File > New Project...`